

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What will be printed?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        print(x)
```

```
    x = 30
```

```
    inner()
```

```
outer()
```

```
A)30
```

```
B)10
```

```
C)NameError
```

```
D)20
```

Q2. What is the output?

```
def fun(x):
```

```
    x += 10
```

```
    return x
```

```
x = 5
```

```
y = fun(x)
```

```
print(x, y)
```

```
A)15 5
```

```
B)5 5
```

```
C)15 15
```

```
D)5 15
```

Q3. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

```
A)The local file can shadow the intended standard-library module
```

```
B)Python automatically renames the local file
```

```
C)Python will always reject it
```

```
D)random cannot be imported
```

Q4. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

```
A)20
```

```
B)None
```

```
C)NameError
```

```
D)10
```

Q5. What is the output?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal x
```

```
        x = x + 5
```

```
        return x
```

```
    inner()
```

```
    return x
```

```
print(outer())
```

```
A)10
```

```
B)5
```

```
C)UnboundLocalError
```

```
D)15
```

Q6. What happens when this function is called?

```
def fun(a=10, b):
```

```
    return a + b
```

```
A)TypeError
```

```
B)NameError
```

```
C)It executes correctly
```

```
D)SyntaxError
```



Q7. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

- A)10
- B)Error
- C)15
- D)5

Q8. if test.py has

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If test.py is executed directly, what will be printed?

- A)A
- B)B
- C)A followed by B
- D)Nothing

Q9. What is the output?

```
try:  
    x = int("abc")  
except ValueError:  
    print("A")  
else:  
    print("B")  
finally:  
    print("C")
```

- A)A
- B)B
- C)A C
- D)B C

Q10. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(  
    map(  
        lambda x: x * 2,  
        filter(lambda x: x % 2 == 0,  
            numbers)  
    )  
)
```

```
print(result)
```

- A)[1, 3, 5]

- B)[2, 4]

- C)[4, 8]

- D)[2, 4, 6, 8, 10]

Q11. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

```
print(result)
```

- A)9
- B)0
- C)10
- D)24

Q12. What is the output?

```
def outer():  
    x = 10  
  
    def inner():  
        return x + 5  
  
    return inner
```

```
f = outer()
```

```
print(f())
```

- A)None
- B)NameError
- C)10
- D)15

Q13. What happens here?

```
try:  
    raise ValueError("Wrong value")  
except ValueError as e:  
    print(e)
```

- A)e
- B)Nothing
- C)Wrong Value
- D)ValueError

Q14. Suppose sample.py contains:

```
x = 10
```



```
def display():  
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)  
sample.display()
```

A)Only Hello

B>Error

C)10

Hello

D)x

display

Q15. What happens?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, a=20))
```

A)SyntaxError

B)30

C)TypeError

D)20

Q16. What is the major difference between the following?

```
import math
```

```
and
```

```
from math import sqrt
```

A)Both import only sqrt

B)First imports only sqrt

C)First provides the module namespace; second imports sqrt directly

D)There is no difference

Q17. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

```
print(fun(30))
```

A)30

B)-10

C)20

D)10

Q18. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is too low")  
    return "Allowed"
```

```
try:  
    print(check_age(15))  
except InvalidAgeError as e:  
    print(type(e).__name__)  
    print(e)
```

A)Nothing, because custom exceptions cannot be caught

B)InvalidAgeError

15

C)InvalidAgeError

Age is too low

D)Exception

Age is too low

Q19. What is the output?

```
def fun(x):  
    if x == 0:  
        return  
    return x + fun(x - 1)
```

```
print(fun(3))
```

A)3

B)6

C)5

D)None

Q20. What will be printed?

```
try:  
    x = int("10")  
except ValueError:  
    print("A")  
else:  
    print("B")  
finally:
```



print("C")

- A)A C
- B)A
- C)B C
- D)B

Q21. What will be the output?

import math

print(math.sqrt(25))

- A)5
- B)5.0
- C)25
- D)sqrt(25)

Q22. What will happen?

```
def fun(a, b=10):  
    return a + b
```

print(fun(b=5, a=20))

- A)TypeError
- B)15
- C)25
- D)SyntaxError

Q23. What will be the output?

x = 10

```
def fun():  
    print(x)  
    x = 20
```

fun()

- A)20
- B)10
- C)NameError
- D)UnboundLocalError

Q24. What does the following function return?

```
def fun():  
    print("Hello")
```

x = fun()

```
print(x)  
A)Hello  
B>Error
```

C)Hello None

D)None

Q25. What will be the output?

```
def fun(x):
```

```
    try:
```

```
        if x < 0:
```

```
            raise ValueError("negative")
```

```
        return x
```

```
    except ValueError:
```

```
        return 100
```

```
    finally:
```

```
        print("cleanup")
```

print(fun(-5))

- A)100
- cleanup
- B)negative
- cleanup
- C)cleanup
- 100
- D)cleanup
- 5

Q26. What is the output?

```
def fun():
```

```
    try:
```

```
        return 10
```

```
    finally:
```

```
        return 20
```

print(fun())

- A)20
- B)None
- C)RuntimeError
- D)10

Q27. What will be printed?

```
def fun():
```

```
    print("A")
```

```
    return
```

```
    print("B")
```

fun()

- A)A
- B)B
- C)Nothing
- D)A B



Q28. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

```
a = fun()  
b = fun()
```

```
print(a)  
print(b)
```

A)Error
B)[10]
[10, 10]
C)[10]
[10]
D)[10, 10]
[10]

Q29. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

A)6.0
B)NameError
C)6
D)36

Q30. What will be the output?

```
x = 10
```

```
def outer():  
    x = 20
```

```
    def inner():  
        global x  
        x = 30
```

```
    inner()  
    print(x)
```

```
outer()  
print(x)
```

A)20
10
B)30
20

C)20

30

D)30

30

Q31. Which object keeps track of modules that have already been imported during the current Python process?

A)__module__

B)sys.path

C)sys.modules

D)sys.imports

Q32. What will happen?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal y  
        y = 20
```

```
    inner()
```

```
outer()
```

A)SyntaxError

B)NameError

C)20

D)10

Q33. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went wrong")
```

```
try:  
    test()  
except MyError as e:  
    print(type(e).__name__)
```

A)TypeError

B)Exception

C)Something went wrong

D)MyError

Q34. What will happen?

```
def fun(n):
```



```

if n == 0:
    return 1
return n * fun(n)

```

print(fun(3))

- A)TypeError
- B)6
- C)RecursionError
- D)3

Q35. What happens here?

```

def fun(a):
    return a

```

print(fun())

- A)TypeError
- B)NameError
- C)None
- D)0

Q36. What is the output?

```

try:
    print(10 / 0)
except Exception:
    print("A")
except ZeroDivisionError:
    print("B")

```

- A)SyntaxError
- B)A B
- C)B
- D)A

Q37. What happens when this code runs?

```

def first():
    second()

def second():
    third()

def third():
    raise RuntimeError("Failure")

```

```

try:
    first()
except RuntimeError:
    print("Handled")

```

A)RuntimeError cannot propagate through functions

- B)NameError
- C)TypeError
- D)Handled

Q38. What will be printed?

```

def fun():
    try:
        return 10
    finally:
        print("Finally")

```

print(fun())

- A)10 followed by Finally
- B)Finally followed by 10
- C)10 only
- D)Finally only

Q39. What is the output?

```

def fun():
    try:
        print("A")
        raise ValueError
    except ValueError:
        print("B")
        return 10
    else:
        print("C")
        return 20
    finally:
        print("D")

```

print(fun())

- A)A
- C
- D
- 20
- B)A
- B
- C
- D
- 10
- C)A
- B
- 10
- D



D)A

B

D

10

Q40. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a
```

```
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

A) ImportError

B) Module loaded

10

C) 10

10

D) Module loaded

Module loaded

10

Q41. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

```
variable = 10
```

```
fun()
```

A) UnboundLocalError

B) 50

C) 10

D) NameError

Q42. What is the output?

```
def fun(x):  
    return x * 2
```

```
a = fun  
print(a(5))
```

A) 10

B) 5

C) func(5)

D) Error

Q43. Why is

try:

```
    print(10 / 0)
```

except Exception:

```
    print("A")
```

except ZeroDivisionError:

```
    print("B")
```

codes's second handler never reached?

A) ZeroDivisionError occurs before Exception

B) except blocks execute randomly

C) ZeroDivisionError is not an exception

D) Exception is a superclass of ZeroDivisionError

Q44. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

A) A and B

B) B

C) Nothing

D) A

Q45. What will be printed?

try:

```
    print(10 / 0)
```

except ZeroDivisionError:

```
    print("A")
```

except Exception:

```
    print("B")
```

A) Nothing

B) A B

C) A

D) B



Q46. What is the purpose of this statement?

```
if __name__ == "__main__":
```

- A)It prevents all imports
- B)It prevents syntax errors
- C)It identifies code intended to run when the file is executed directly
- D)It creates the main function

Q47. What will be printed?

```
def make_counter():  
    count = 0
```

```
    def counter():  
        nonlocal count  
        count += 1  
        return count
```

```
    return counter
```

```
a = make_counter()  
b = make_counter()
```

```
print(a())  
print(a())  
print(b())  
print(a())
```

A)0

1

0

2

B)1

2

1

3

C)1

1

1

1

D)1

2

3

4

Q48. What will be printed?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, b=20))
```

A)SyntaxError

B)20

C)TypeError

D)30

Q49. What is the output?

```
try:
```

```
    raise ValueError
```

```
except ValueError:
```

```
    print("A")
```

```
    raise
```

A)A, followed by propagation of the same exception

B)B

C)No Output

D)A, and execution ends normally

Q50. What will normally happen?

```
import math
```

```
import math
```

A)The module is normally loaded once and reused

B)SyntaxError

C)ImportError

D)The module is normally loaded and executed twice

